Lista zadań nr 5 16, 18 listopada 2026 r.

zaawansowane techniki C++ i STL

kolekcje

Instytut Informatyki
Uniwersytetu Wrocławskiego **Paweł Rzechonek**

Zadanie 1 (2 pkt)

Napisz prosty *lekser*, który przeanalizuje zadany tekst i podzieli go na tokeny (leksemy) takie jak liczby, identyfikatory, znaki interpunkcyjne, nawiasy, operatory arytmetyczne i relacyjne itp. Zdefiniuj funkcję, która otrzyma tekst w postaci napisu typu `string`, podzieli go na leksemy i zwróci kolekcję zakresów dla kolejnych tokenów występujących w tym napisie `forward_list<span<char>>`. Rzetelnie przetestuj działanie leksera na różnych i skomplikowanych napisach.

Zadanie 2 (4 pkt)

Zaprogramuj *algorytm stacji rozrządowej* Dijkstry, który przekształca wyrażenie arytmetyczne z postaci infiksowej z nawiasami do postaci postfiksowej bez nawiasów (Odwrotna Notacja Polska). Wykorzystaj do tego celu kolekcje standardowe.

W pierwszym kroku należy przekształcić wyrażenie infiksowe zapisane w łańcuchu znakowym `string` na ciąg symboli i umieścić je w kolekcji `forward_list<symbol>` (wykorzystaj poprzednie zadanie). Sama klasa `symbol` ma być tuplą/krotką składającą się z następujących elementów:

- wartość typu wyliczeniowego opisująca rodzaj symbolu (wyliczenie zdefiniuj za pomocą `enum class`),
- opcjonalna wartość liczbowa typu `double` związana z symbolem (dla zwykłych literałów liczbowych albo dla zdefiniowanych stałych jak π czy e),
- opcjonalna nazwa typu `string` (dla stałych, zmiennych, funkcji i operatorów arytmetycznych),
- priorytet typu `int` dla operatorów arytmetycznych i funkcji.

Stałe przechowuj w uporządkowanej kolekcji par `map<string, double>`. Zmienne przechowuj w nieuporządkowanej kolekcji par `unordered_map<string, double>`. Nazwy operatorów i funkcji przechowuj w kontenerze sekwencyjnym `deque<string>`. Kolekcje te będą potrzebne do weryfikacji poprawności zapisu wyrażenia.

Funkcja przekształcająca naturalny zapis infiksowy wyrażenia na postać postfiksową (algorytm stacji rozrządowej Dijkstry) dostaje jako argument `forward_list<symbol>` (wyrażenie w zapisie infiksowym) i zwraca jako wynik także `forward_list<symbol>` (wyrażenie w zapisie postfiksowym). Algorytm Dijkstry do przekształcenia wyrażenia powinien korzystać ze stosu symboli `stack<symbol>`.

Zadanie 3 (4 pkt)

Zdefiniuj strukturę danych do przetrzymywania *ważonego grafu dynamicznego* w postaci list sąsiadów. Wspomniana dynamika grafu oznacza, że można dodawać nowe wierzchołki oraz usuwać istniejące wierzchołki z grafu. Wierzchołki w grafie niech posiadają swoje unikalne identyfikatory typu `string` (identyfikator to niepusty napis złożony tylko z liter). Co się tyczy krawędzi, to można je dodawać i usuwać, a także zmieniać ich wagę. Każda krawędź w grafie ma przypisaną wagę typu `double` (wagi mogą być ujemne). Wykorzystaj do tego celu kontenery z STL.

Przetestuj swoją implementację grafu definiując funkcję, która wyznaczy MST (minimalne drzewo rozpinające) w zadanym grafie. Wynikiem działania tej funkcji także ma być graf (będący drzewem). Zaprogramuj w tym celu *algorytm Prima*, wykorzystując kolejkę priorytetową wierzchołków osiągalnych ze zbudowanego dotychczas MST.